

Demo Guide

FINE-TUNING SESSION

User Feedback to Engineering Insights

Section 1: What We Are Building

In this session, you will build a 2-agent pipeline in n8n that takes raw user feedback (app reviews, support tickets, NPS comments) and converts it into structured engineering insights. You will then evaluate the pipeline using DeepEval, fine-tune both agents separately, and measure the improvement.

The Pipeline

```
User Feedback --> Feedback Classifier (Agent 1) --> Engineering Insight Writer (Agent 2) --> Engineering Ticket
```

Agent 1 - Feedback Classifier: Reads raw user feedback and extracts technical signals: category (bug, feature request, performance issue, unclear), severity, affected system, platform, environment, exact key phrases, reproduction hints, and missing technical context.

Agent 2 - Engineering Insight Writer: Takes the classification and produces a structured engineering ticket with: title, ticket type, technical summary, affected components, reproduction steps, investigation checklist, data gaps, and priority recommendation.

The Problem We Are Solving

When the base model (meta-llama/llama-3.1-8B-Instruct) receives vague feedback like 'This app is garbage', it tends to invent technical details: fabricated system names ('API Gateway'), made-up reproduction steps, and imaginary root causes. This is hallucination -- the model generates confident, professional-looking engineering tickets from nothing.

Fine-tuning teaches each agent the correct behavior: the Classifier learns to flag vague feedback as UNCLEAR with AFFECTED_SYSTEM set to UNKNOWN. The Insight Writer learns to output 'Insufficient feedback for engineering action' and recommend reaching out to the user instead of fabricating a ticket.

The Evaluation Loop

Before fine-tuning: Run 8 test feedback items (4 detailed + 4 vague) through the pipeline, collect outputs, and evaluate with DeepEval. Save the scores.

After fine-tuning: Replace both models in n8n with the fine-tuned versions, re-run the same 8 items, and evaluate again. Compare the scores to prove the improvement.

Section 2: Understanding the Files

You will receive 9 files. Here is what each one does and why it exists.

File	Purpose
Fine Tuning n8n - using local LLM.json https://sme1.app.n8n.cloud/workflow/B9ilZqdqz20mT63H	The n8n workflow. Contains both agents and their prompts. You import this into n8n.
training_data_classifier.jsonl	15 training examples for the Feedback Classifier. Each example shows the correct classification for a specific feedback item. Used by the fine-tuning script.
training_data_insight.jsonl	15 training examples for the Engineering Insight Writer. Each example shows the correct engineering ticket for a classified feedback item. Used by the fine-tuning script.
test_feedback_engineering.py	The DeepEval evaluation script. Contains 8 test cases and 4 evaluation metrics. You paste your n8n outputs into this file and run it to get scores.
download_hf_model.py	To download the base model (meta-llama/Llama-3.1-8B-Instruct) from HuggingFace
load_run_local_model.py	Load the base model and run the inference endpoint at local machine
fine_tune_training.py	The fine-tuning script. Fine-tune the model using training data and save the new model parameters (weights) to the local machine.
merge_to_base_model.py	Create new self-sufficient model (fine-tuned) by merging the new weights to the base model.
load_run_local_finetune_model.py	Load both the fine-tuned models and run the inference endpoint at local machine

Why JSONL Files for Training

OpenAI's fine-tuning API requires training data in JSONL (JSON Lines) format. Each line is a complete conversation example with three messages: a system prompt (the agent's role), a user

message (the input), and an assistant message (the desired output). The model learns from these examples to produce similar outputs.

The training data contains 10 good examples (detailed feedback with grounded outputs) and 5 corrected examples (vague feedback with the correct 'insufficient data' response). The corrected examples are the key to fixing hallucination: they teach the model what to do when it does not have enough information.

Models Used in This Session

Purpose	Model
n8n agents (Classifier LLM and Insight LLM nodes)	meta-llama/Llama-3.1-8B-Instruct
Fine-tuning base model (in fine_tune_training.py)	meta-llama/Llama-3.1-8B-Instruct
DeepEval evaluation judge (in test_feedback_engineering.py)	gpt-4o

You need one OpenAI API key. The evaluation model (gpt-4o) is accessible with this key and is only used twice during the entire session (before and after fine-tuning), so the cost is minimal. Using gpt-4o-mini as the judge would produce noisy, unreliable scores.

Why DeepEval

DeepEval uses GPT-4 as a judge to evaluate your pipeline's outputs against specific criteria you define. It is like having a senior engineer review every output and score it. You define the criteria in plain English (such as 'does the response invent system names not mentioned in the feedback?'), and DeepEval applies that criteria consistently across all test cases.

We use 4 metrics: Engineering Groundedness (does the output stay truthful to the input?), Engineering Usefulness (would an engineer find this helpful?), Vague Feedback Handling (does the pipeline correctly flag vague input?), and Answer Relevancy (is the output relevant to the input?).

Section 3: Setup

Step 3.1: Prerequisites

- n8n installed and running (cloud or self-hosted)
- OpenAI API key with fine-tuning access (from platform.openai.com)
- Python 3.9 or higher installed
- VS Code or any code editor

Setting Up n8n

n8n Cloud (easiest): Go to <https://app.n8n.cloud> and sign up for a free account. You will get a hosted n8n instance with no installation needed.

Please watch these videos to setup VS code on your windows/mac device:

MacOS - <https://www.youtube.com/watch?v=7PIHlkP2WQ>

Windows - <https://www.youtube.com/watch?v=zK1rKC7QVwk>

Please watch these videos to setup Python on your windows/mac device:

MacOS - https://youtu.be/VJjeMdl6yWA?si=rF9_j844MoXPXwfo

Windows - https://youtu.be/8mO6QXOcpqU?si=Eg5_ZqEjl1aZcWbk

Step 3.2: Install Python Packages

Open the terminal (Ctrl + backtick on Windows, Cmd + backtick on Mac) and create a new folder called 'fine-tuning-demo'.

```
pip install pip install huggingface-hub torch transformers fastapi  
typing-extensions typing-inspection peft trl datasets deepeval  
openai python-dotenv
```

If pip is not found, try: `pip3 install pip install huggingface-hub torch transformers fastapi typing-extensions typing-inspection peft trl datasets deepeval openai python-dotenv`

Verify the installation worked:

```
python -c "from openai import OpenAI; from deepeval import  
evaluate; print('All packages installed successfully')"
```

(On Mac, use `python3` instead of `python`)

If you see "All packages installed successfully", you are good. If you see an error about a missing module, run the pip install again.

Step 3.3: Set Your HuggingFace & OpenAI API Key

Set the key as an environment variable. Do NOT hardcode it in the Python files.

In the file called `.env` in your fine-tuning-demo folder store your API key securely so you do not have to hardcode it in the Python scripts.

If it's not there, In VS Code, right-click in the sidebar and select "New File". Name it `.env` (with the dot at the start). Inside the file, add exactly one line:

```
OPENAI_API_KEY=<YOUR_OPENAI_KEY>
```

```
HF_TOKEN=<YOUR_HF_KEY>
```

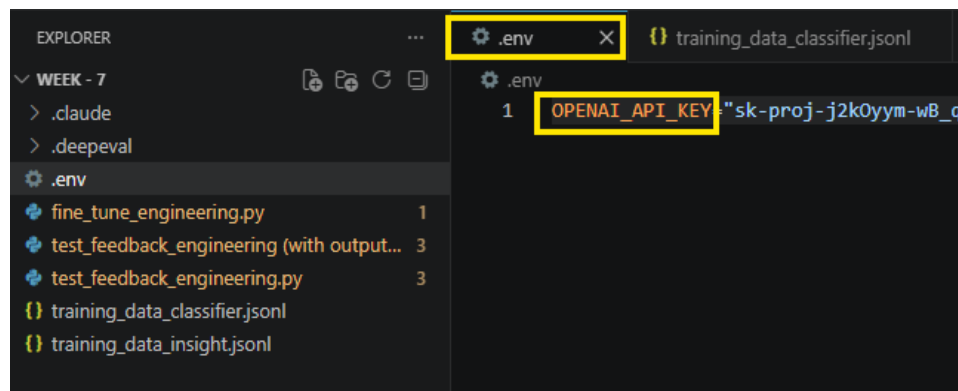
Replace `<YOUR_OPENAI_KEY>` & `<YOUR_HF_KEY>` with your actual OpenAI API and HuggingFace keys. Save the file.

The Python scripts use the `python-dotenv` package (installed in Step 3.2) to read this file automatically. You do not need to set environment variables manually in the terminal.

Important: Never share your `.env` file. Never commit it to GitHub. It contains your private API key.

Verify it works: In the terminal, run:

```
python -c "from dotenv import load_dotenv; load_dotenv(); import os; key=os.environ.get('OPENAI_API_KEY',''); print('Key loaded' if key.startswith('sk-') else 'Key NOT found - check your .env file')"
```



Important:

This key is set only for the current terminal session. If you close and reopen the terminal, you need to set it again.

Step 3.4: Place All Files

Put all 10 files into your 'fine-tuning-demo' folder:

```
fine-tuning-demo/  
download_hf_model.py  
load_run_local_model.py  
fine_tune_training.py  
merge_to_base_model.py  
load_run_local_finetune_model.py  
test_feedback_engineering.py  
training_data_classifier.jsonl  
training_data_insight.jsonl  
.env
```

Section 4: Download and Run LLM Locally

1. Visit HuggingFace and request access to the Llama model

Why Access Is Required:

License Agreement: Meta requires users to accept licensing terms before downloading or deploying the model.

Repository Unlocking: Llama models are gated on Hugging Face and require approval before download.

Token Authorization: Approval enables Hugging Face access tokens to authenticate model downloads and usage.

2. Visit the model page at <https://huggingface.co/meta-llama/Llama-3.1-8B-Instruct>
3. Review the license agreement
4. Complete the required form
5. Submit and wait for approval.
6. Download the Base Model (download time: ~15 min)

```
python download_hf_model.py
```

7. Run the Local Base Model

```
python load_run_local_model.py
```

8. Install & Configure Using ngrok

Install & setup ngrok: <https://dashboard.ngrok.com/get-started/setup/mac-os>

9. Expose the base model inference endpoint to public internet.

```
ngrok http --host-header=rewrite http://127.0.0.1:8005
```

Note the url exposed by ngrok, e.g. <https://spinner-tighten-jeep.ngrok-free.dev>

```
ngrok
One gateway for every AI model. Available in early access *now*: https://ngrok.com/r/ai

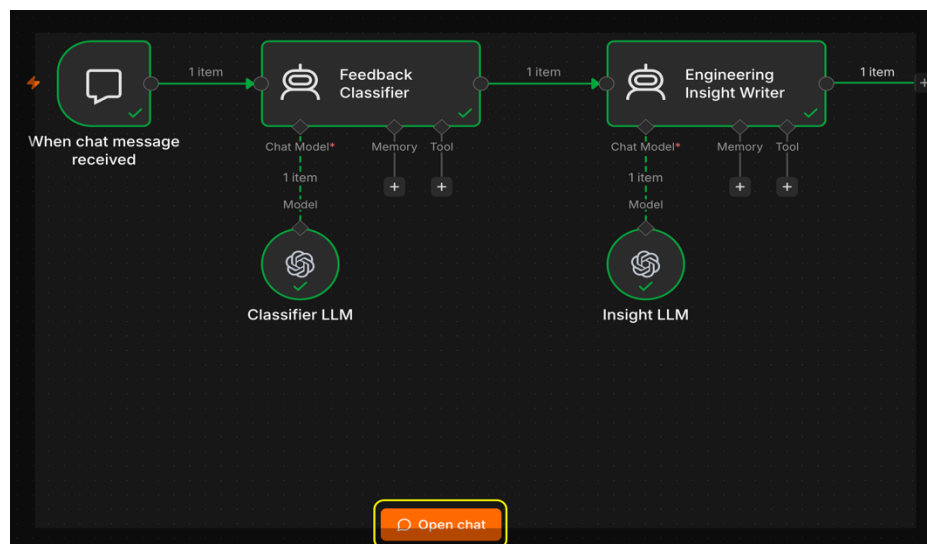
Session Status      online
Account             Joy (Plan: Free)
Update              update available (version 3.39.6, Ctrl-U to update)
Version             3.39.4
Region              United States (us)
Latency             43ms
Web Interface       http://127.0.0.1:4040
Forwarding           https://spinner-tighten-jeep.ngrok-free.dev -> http://localhost:8005

Connections
  ttl  opn  rt1  rt5  p50  p90
   31    0   0.00 0.00 29.74 48.06

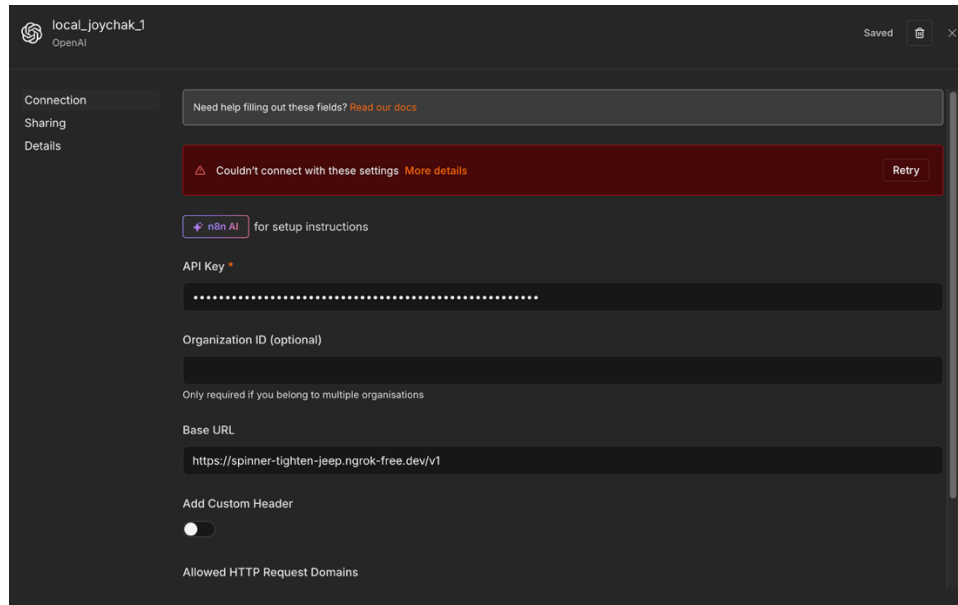
HTTP Requests
```

Section 5: Import the n8n Workflow

1. Open n8n in your browser
2. Click '+' to create a new workflow
3. Click the three-dots menu (top right) and select 'Import from File'
4. Select "Fine Tuning n8n - using local LLM.json"



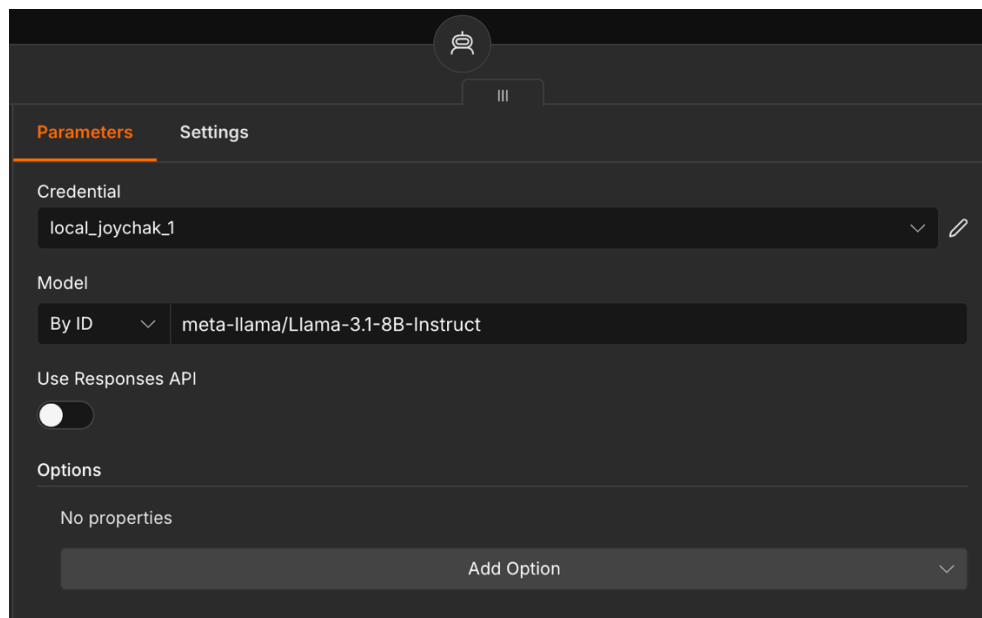
5. Click the 'Classifier LLM' node. You need to add your local model credential:
 - a. Click the "Credential" dropdown inside the node
 - b. Click "Create New Credential"
 - c. In the dialog, populate the **Base URL** as <ngrok_rul/v1>, e.g., <https://spinner-tighten-jeep.ngrok-free.dev/v1> and any value in the **API Key**.



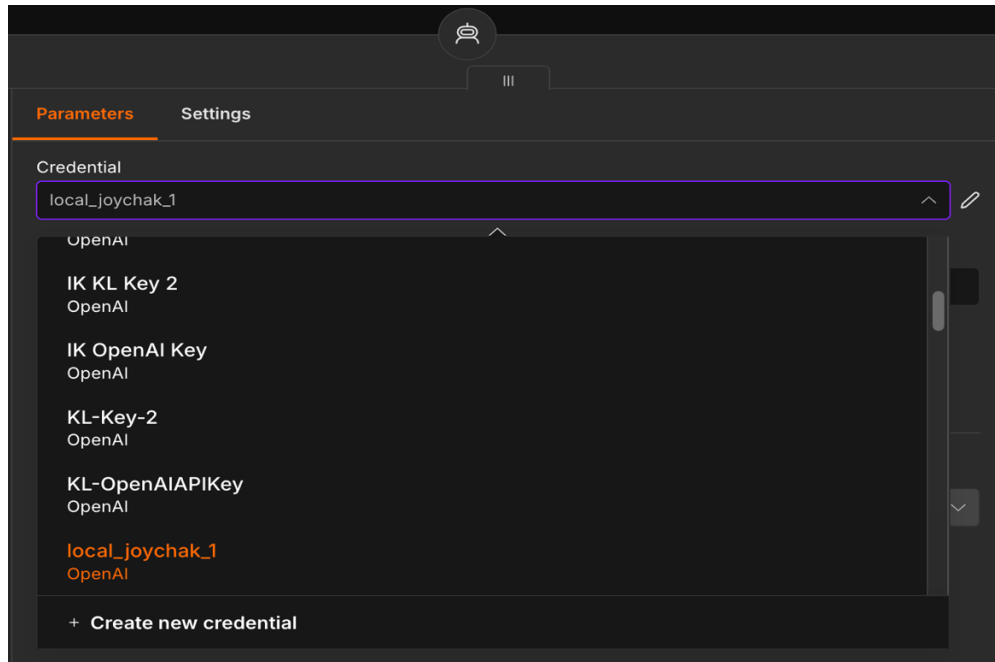
'Couldn't connect with these settings' Error:

Please ignore this error while creating credential for local model connection.

- d. Click "Save"
- e. Set the Model field for "By ID", where the value is set to **meta-llama/Llama-3.1-8B-Instruct**



6. Click the 'Insight LLM' node. Select the same credential you just created (it will appear in the dropdown). Verify model id is **meta-llama/Llama-3.1-8B-Instruct**.



7. Save the workflow

Checkpoint:

Click the Chat button (bottom right). Send 'test' as a message. You should see output from the Engineering Insight Writer within 30-60 seconds. If you get an error, verify your local model inference endpoint credential is correct.

If import does not work:

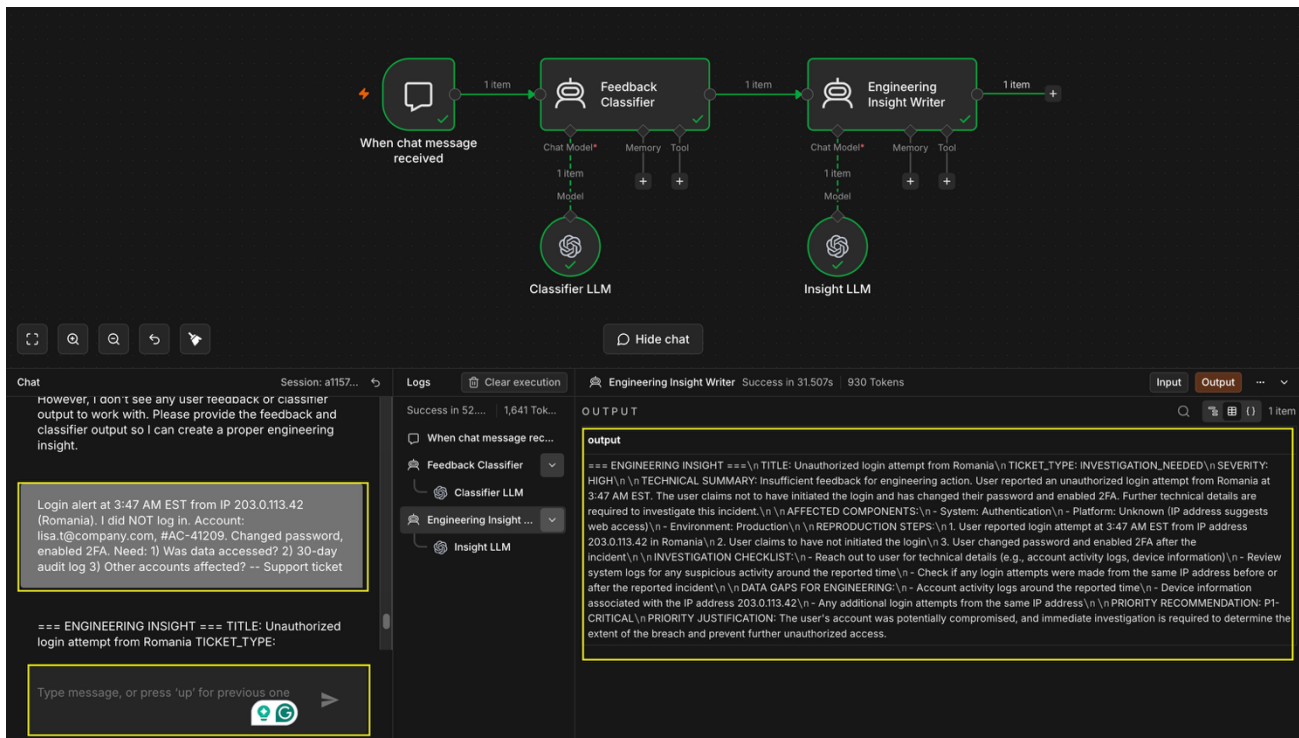
Build manually: add Chat Trigger, add two Agent nodes (Feedback Classifier and Engineering Insight Writer), add two OpenAI Chat Model sub-nodes (one for each agent). Connect Chat Trigger to Classifier, Classifier to Insight Writer. Copy the system prompts from the JSON file into each agent's System Message field.

Section 6: Run Test Feedback Items

Run each of the 8 feedback items below through the n8n chat one at a time. Wait for each to complete (30-60 seconds). Copy and save each output -- you will need them later.

Detailed Feedback (4 items -- should produce good engineering insights)

F1: The checkout page keeps freezing when I try to apply a discount code on my iPhone 14. Happens every time. I've lost 3 orders this week because of it. Using Safari, latest iOS. -- App Store review, 1 star



F2: The API response times have degraded significantly since last month. Our p95 latency went from 200ms to 800ms. We're on the Enterprise plan and this is affecting our production systems. We've opened 3 tickets about this already. Account: api-team@bigcorp.com -- Support ticket

F3: The onboarding flow is confusing. I signed up yesterday and still can't figure out how to create my first project. The getting started guide links to a 404 page. I watched 3 YouTube tutorials and they all show a different UI than what I see. Free plan, Chrome on Windows. -- Support ticket

F4: Login alert at 3:47 AM EST from IP 203.0.113.42 (Romania). I did NOT log in. Account: lisa.t@company.com, #AC-41209. Changed password, enabled 2FA. Need: 1) Was data accessed? 2) 30-day audit log 3) Other accounts affected? -- Support ticket

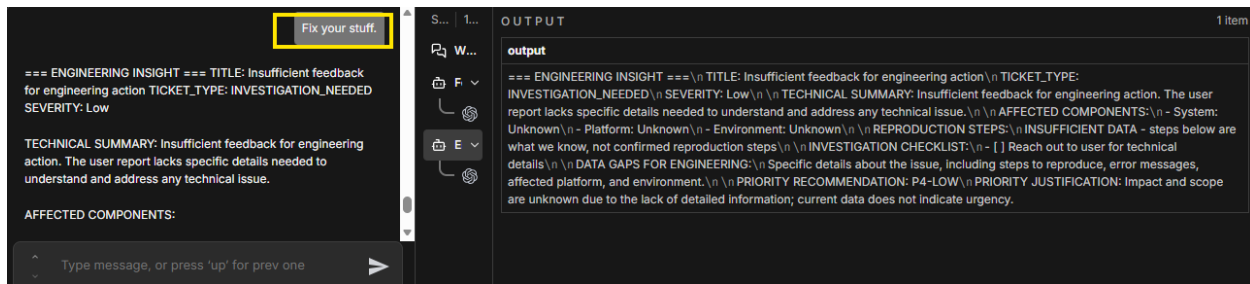
Vague Feedback (4 items -- should be flagged as unactionable)

F5: This app is garbage. Total waste of money.

F6: Terrible. Just terrible. 0 stars if I could.

F7: Meh. It's fine I guess.

F8: Fix your stuff.



What to look for in vague outputs:

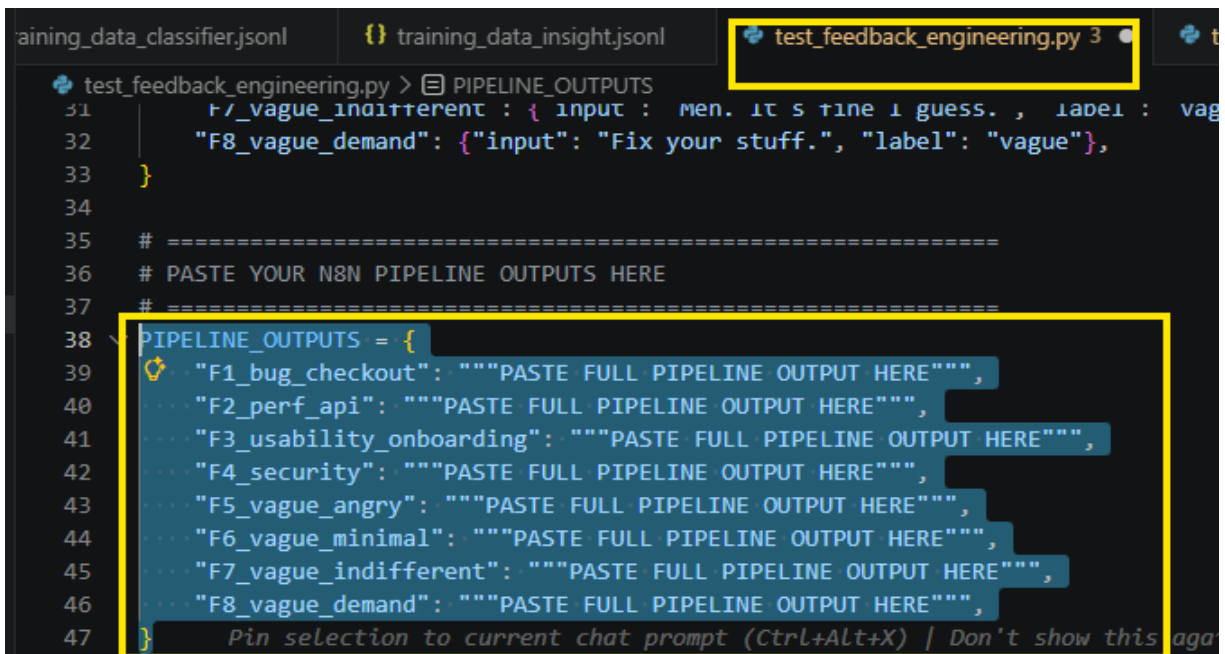
Check F5-F8 outputs. Does the Insight Writer invent system names, reproduction steps, or root causes? Or does it correctly say 'insufficient feedback for engineering action'? Write down anything it invented -- this is the hallucination you will fix with fine-tuning.

Checkpoint:

You should have 8 saved outputs. Keep them in a text file or document. You will paste them into the evaluation script next.

Section 7: Evaluate with DeepEval (Before Fine-Tuning)

1. Open test_feedback_engineering.py in VS Code
2. Find the PIPELINE_OUTPUTS section (around line 40)
3. Replace each 'PASTE FULL PIPELINE OUTPUT HERE' with the actual output from n8n for that feedback item (Refer Appendix 1).
4. Save the file



```
training_data_insight.jsonl  test_feedback_engineering.py 3  test_feedback_engineering
test_feedback_engineering (with outputs from n8n).py > ...
32 |         "F8_vague_demand": { "input": "I'm your stuff, I love it. Vague.",
33 |     }
34 |
35 | # =====
36 | # PASTE YOUR N8N PIPELINE OUTPUTS HERE
37 | # =====
38 | PIPELINE_OUTPUTS = {
39 |     "F1_bug_checkout": "" == ENGINEERING INSIGHT == ...
88 |     "F2_perf_api": "" == ENGINEERING INSIGHT == ...
148 |     "F3_usability_onboarding": "" == ENGINEERING INSIGHT == ...
208 |     "F4_security": "" == ENGINEERING INSIGHT == ...
270 |     "F5_vague_angry": "" == ENGINEERING INSIGHT == ...
313 |     "F6_vague_minimal": "" == ENGINEERING INSIGHT == ...
363 |     "F7_vague_indifferent": "" == ENGINEERING INSIGHT == ...
406 |     "F8_vague_demand": "" == ENGINEERING INSIGHT == ...
445 | }
446 |
```

In the terminal (make sure your API key is still set):

```
python test_feedback_engineering.py
```

Note: The evaluation takes 1-3 minutes. DeepEval calls gpt-4o as a judge to score each output against the criteria defined in the metrics. This is a different, smarter model than the gpt-4o-mini used in your n8n pipeline. Using a smarter judge model ensures accurate and consistent scoring.

```
PS C:\Users\LENOVO\Downloads\week - 7> python test_feedback_engineering.py
=====
FEEDBACK TO ENGINEERING INSIGHTS - DeepEval Evaluation
=====

=====
EVALUATING DETAILED FEEDBACK
(should produce grounded, useful engineering insights)
=====
No detailed feedback outputs pasted yet.

=====
EVALUATING VAGUE FEEDBACK
(should flag as unactionable, not invent engineering details)
=====
No vague feedback outputs pasted yet.

=====
EVALUATION COMPLETE
=====

What to look for:
Detailed: Engineering Groundedness + Usefulness should PASS
Vague:    Vague Feedback Handling should PASS
All:      Relevancy should PASS
```

Understanding the Results

Metric	What It Checks
Engineering Groundedness	Does the output ONLY use info from the original feedback? Fails if system names, error codes, or details are invented.
Engineering Usefulness	Would an engineer find this helpful? Checks if investigation checklist and data gaps are reasonable.
Vague Feedback Handling	For vague inputs: does it correctly flag as unactionable instead of inventing a ticket?
Answer Relevancy	Is the output relevant to the input feedback?

Expected results before fine-tuning:

Some detailed tickets will FAIL on Groundedness because the base gpt-4o-mini model invents system names (like 'API Gateway' when the user only said 'API'). Vague tickets should mostly PASS on Vague Handling if the model correctly identifies them as unclear. However, you may notice the base model assigns inconsistent severity levels (mixing S1/S2/S3 scales with P2/P3 scales) and sometimes marks vague input as S1-CRITICAL when it should be LOW. These are exactly the kinds of errors fine-tuning will fix.

Rate limit error:

If you get a rate limit error, wait 60 seconds and run again. DeepEval uses GPT-4 as judge, which has rate limits.

'insufficient_quota' error:

Your OpenAI account needs billing credits. Add payment at platform.openai.com/account/billing.

Section 8: Fine-Tune Both Models

This is where the model learns from the training data. The fine-tuning script trains TWO models in parallel: one for the Feedback Classifier and one for the Engineering Insight Writer.

What the Training Data Teaches

Classifier training (15 examples): How to extract exact quotes as KEY_PHRASES (not paraphrase), when to flag feedback as UNCLEAR with AFFECTED_SYSTEM set to UNKNOWN, and what fields to include for engineering triage.

Insight Writer training (15 examples): How to write structured engineering tickets without inventing data, how to handle UNCLEAR classifications by saying 'Insufficient feedback for

engineering action', and how to keep frequency signal as 'single report' instead of fabricating statistics.

Run Fine-Tuning

This is a 2 steps process:

1. Train the LoRA Adapters: Finetune the base model and save the updated model parameters.

In the terminal:

```
python fine_tune_training.py
```

The script will:

1. Validate both JSONL files (15 examples each)
2. Create two fine-tuning jobs
3. Load the weights of the base model and training data from JSONL file
4. Train the base model using samples from the JSONL files
 - 4.1. Training time depends on number of iteration (max_steps value in code). Training for 100 steps will take ~2-3 hr for each model.
 - 4.2. Every 10 steps, the code will provide log update.
5. Save the updated model parameters (e.g. weights) to local directory

```
(hf-project-demo) [joychak@ ~/joy/code/repository/IK/personalization_fine_tuning_of_agents_module/Live_Class_Slides_and_Project/hf-project-demo/code] python fine_tune_training.py
Note: Environment variable HF_TOKEN is set and is the current active token independently from the token you've just configured.
Loading weights: 15% | 2/291 [00:02:06:44, 1.40s/it]
/Users/joychak/joy/code/repository/IK/personalization_fine_tuning_of_agents_module/Live_Class_Slides_and_Project/hf-project-demo/.venv/lib/python3.11/site-packages/bitsandbytes/backends/default/ops.py:223: FutureWarning: _check_is_size will be removed in a future PyTorch release along with guard_size_oblivious. Use _check(i >= 0) instead.
  torch._check_is_size(blocksize)
Loading weights: 100% | 291/291 [00:49:00:00, 5.86it/s]

Base model loaded with 4-bit quantization successfully!

Uploading /Users/joychak/joy/code/repository/IK/personalization_fine_tuning_of_agents_module/Live_Class_Slides_and_Project/hf-project-demo/code/training_data_classifier.jsonl...
Map: 100% | 15/15 [00:00:00:00, 483.95 examples/s]

Total training examples: 15
First sample data: {'messages': [{'role': 'system', 'content': 'You are a user feedback classifier for engineering teams. Extract technical signals. KEY_PHRASES must be exact quotes. REPRODUCTION_HINTS must only contain user-described behavior. If feedback is vague, set CATEGORY to UNCLEAR and AFFECTED_SYSTEM to UNKNOWN. Do NOT invent technical details.'}, {'role': 'user', 'content': 'Can you add Salesforce integration? We need to sync our customer records bi-directionally. Currently we export CSV manually every week which takes our ops team 4 hours. We're on Enterprise, 340 users. Would pay extra for this. Account: ops@megacorp.com -- Feature request form'}, {'role': 'assistant', 'content': 'CATEGORY: INTEGRATION_REQUEST\nSEVERITY: MEDIUM\nAFFECTED_SYSTEM: UNKNOWN\nPLATFORM: UNKNOWN\nENVIRONMENT: Enterprise plan, 340 users\nSENTIMENT: NEUTRAL\nKEY_PHRASES: ["Salesforce integration", "sync customer records bi-directionally", "export CSV manually every week", "takes our ops team 4 hours", "Would pay extra"]\nREPRODUCTION_HINTS: [Not a bug. Current workaround is weekly manual CSV export taking 4 hours.]\nMISSING_TECHNICAL_CONTEXT: [Salesforce objects and fields to sync, record volume, acceptable sync latency, whether REST or bulk API is preferred, data format requirements]'}], 'text': '<|begin_of_text|><|start_header_id|>system<|end_header_id|>\n\nCutting Knowledge Date: December 2023\n\nToday Date: 26 Jul 2024\n\nYou are a user feedback classifier for engineering teams. Extract technical signals. KEY_PHRASES must be exact quotes. REPRODUCTION_HINTS must only contain user-described behavior. If feedback is vague, set CATEGORY to UNCLEAR and AFFECTED_SYSTEM to UNKNOWN. Do NOT invent technical details.<|eot_id|><|start_header_id|>user<|end_header_id|>\n\nCan you add Salesforce integration? We need to sync our customer records bi-directionally. Currently we export CSV manually every week which takes our ops team 4 hours. We're on Enterprise, 340 users. Would pay extra for this. Account: ops@megacorp.com -- Feature request form<|eot_id|><|start_header_id|>assistant<|end_header_id|>\n\n\nCATEGORY: INTEGRATION_REQUEST\nSEVERITY: MEDIUM\nAFFECTED_SYSTEM: UNKNOWN\nPLATFORM: UNKNOWN\nENVIRONMENT: Enterprise plan, 340 users\nSENTIMENT: NEUTRAL\nKEY_PHRASES: ["Salesforce integration", "sync customer records bi-directionally", "export CSV manually every week", "takes our ops team 4 hours", "Would pay extra"]\nREPRODUCTION_HINTS: [Not a bug. Current workaround is weekly manual CSV export taking 4 hours.]\nMISSING_TECHNICAL_CONTEXT: [Salesforce objects and fields to sync, record volume, acceptable sync latency, whether REST or bulk API is preferred, data format requirements]<|eot_id|>'}

LoRA adapters added to the model successfully!

Training arguments set successfully!

Tokenizing train dataset: 100% | 15/15 [00:00:00:00, 1038.06 examples/s]

Starting training...
[Transformers] The tokenizer has new PAD/BOS/EOS tokens that differ from the model config and generation config. The model config and generation config were aligned accordingly, being updated with the tokenizer's values. Updated tokens: {'eos_token_id': 128009, 'pad_token_id': 128009}.
0% | 0/2 [00:00:00:00, 71t/s]
/Users/joychak/joy/code/repository/IK/personalization_fine_tuning_of_agents_module/Live_Class_Slides_and_Project/hf-project-demo/.venv/lib/python3.11/site-packages/torch/utils/data/dataloader.py:752: UserWarning: 'pin_memory' argument is set as true but not supported on MPS now, device pinned memory won't be used.
  super().__init__(loader)
/Users/joychak/joy/code/repository/IK/personalization_fine_tuning_of_agents_module/Live_Class_Slides_and_Project/hf-project-demo/.venv/lib/python3.11/site-packages/bitsandbytes/backends/default/ops.py:314: FutureWarning: _check_is_size will be removed in a future PyTorch release along with guard_size_oblivious. Use _check(i >= 0) instead.
  torch._check_is_size(blocksize)
{'train_runtime': '100.8s', 'train_samples_per_second': '0.147', 'train_steps_per_second': '0.018', 'train_loss': '3.058', 'entropy': '1.713', 'num_tokens': '4327', 'mean_token_accuracy': '0.518', 'epoch': '1'}
100% | 2/2 [01:48:00:00, 54.42s/it]

Training complete and model saved!

Saving LoRA adapters separately...
LoRA adapters saved to ./Llama-3.1-8B-Instruct-FineTuned-Classifier-v2-adapters successfully!
```

2. Merge the Adapters to the Base Model: Load the base model, merge the parameter, save the updated model in local machine, and publish to HuggingFace (optionally)

In the terminal:

```
python merge_to_base_model.py
```

The script will:

1. Load the new LoRA trained weights
2. Load the base model and update the weights
3. Save the updated model to local machine, i.e., this model can be used as a self-sufficient model.
4. Optionally, can publish to the HuggingFace.
5. This is usually fast and takes 1-2 min per model.

```
(HF_project_demo) [joyche@ ~/joy/code/repository/IK/personalization_fine_tuning_of_agents_module/live_class_slides_and_project/HF_project_demo/code] python merge_to_base_model.py
Note: Environment variable HF_TOKEN is set and is the current active token independently from the token you've just configured.

Starting merging process for Classifier model...
[transformers] torch.dtype is deprecated! Use dtype instead!
Loading weights: 100% | 291/291 [00:05:00:00, 57.371t/s]

Base model and Fine-tuned adapters loaded successfully for merging!

Merging LoRA adapters into the base model...

LoRA adapters merged into the base model successfully!

Saving the merged model locally...
Writing model shards: 100% | 4/4 [00:06:00:00, 1.56s/it]

Merged model saved locally at ./Llama-3.1-8B-Instruct-FineTuned-Classifier-v2 successfully!

Pushing the merged model to Hugging Face Hub...
Writing model shards: 100% | 4/4 [00:03:00:00, 1.081t/s]
Processing Files (4 / 4) : 100%
New Data Upload : 100%
...0004-of-00004.safetensors: 100% | 1.19GB / 1.19GB | 5.00GB / 5.00GB | 4.92GB / 4.92GB | 4.95GB / 4.95GB
...0002-of-00004.safetensors: 100% | 1.19GB / 1.19GB | 5.00GB / 5.00GB | 4.92GB / 4.92GB | 4.95GB / 4.95GB
...0003-of-00004.safetensors: 100% | 1.19GB / 1.19GB | 5.00GB / 5.00GB | 4.92GB / 4.92GB | 4.95GB / 4.95GB
...0001-of-00004.safetensors: 100% | 1.19GB / 1.19GB | 5.00GB / 5.00GB | 4.92GB / 4.92GB | 4.95GB / 4.95GB
README.md: 100% | 17.2MB / 17.2MB | 777B/s | 17.2MB / 17.2MB | 777B/s
Processing Files (1 / 1) : 100%
New Data Upload : 100%
...mpv/v1/tokenizer.json: 100% | 17.2MB / 17.2MB | 777B/s | 17.2MB / 17.2MB | 777B/s

Merged model pushed to Hugging Face Hub successfully!

Total merging and saving time: 89.26 seconds (1.49 minutes)
Merging Classifier model process completed successfully!
```

Section 9: Run the fine-tuned models locally

1. Run the Local fine-tuned models

```
python load_run_local_finetune_model.py
```

2. Expose the fine-tuned model inference endpoint to public internet.

```
ngrok http --host-header=rewrite http://127.0.0.1:8006
```

Note the url exposed by ngrok, e.g. <https://spinner-tighten-jeep.ngrok-free.dev>

Section 10: Connect Fine-Tuned Models to n8n

1. Open your n8n workflow in the browser
2. Click the 'Classifier LLM' node. You need to add a new local model credential:
 - a. Click the "Credential" dropdown inside the node
 - b. Click "Create New Credential"
 - c. In the dialog, populate the **Base URL** as <ngrokrul/v1>, e.g., <https://spinner-tighten-jeep.ngrok-free.dev/v2> and any value in the **API Key**.
 - d. Click "Save"

- e. Set the Model field for “By ID”, where the value is set to **Llama-3.1-8B-Instruct-FineTuned-Classifier-v1**

API Key *

Organization ID (optional)

Only required if you belong to multiple organisations

Base URL

https://spinner-tighten-jeep.ngrok-free.dev/v2

Add Custom Header

Allowed HTTP Request Domains

Parameters Settings

Credential

local_joychak_2

Model

By ID Llama-3.1-8B-Instruct-FineTuned-Classifier-v1

Use Responses API

Options

No properties

Add Option

3. Click the 'Insight LLM' node. You need to add a new local model credential:
 - a. Click the "Credential" dropdown inside the node
 - b. Click "Create New Credential"
 - c. In the dialog, populate the **Base URL** as <ngrok_rul/v3>, e.g., <https://spinner-tighten-jeep.ngrok-free.dev/v3> and any value in the **API Key**.
 - d. Click "Save"
 - e. Set the Model field for “By ID”, where the value is set to **Llama-3.1-8B-Instruct-FineTuned-InsightWriter-v1**
4. Save the workflow

'Couldn't connect with these settings' Error:

Please ignore this error while creating credential for local model connection.

Checkpoint:

Send 'test' in the n8n chat to verify the fine-tuned models are working. You should get a response within 30-60 seconds.

Section 11: Evaluate After Fine-Tuning

1. Run all 8 feedback items through the n8n chat again (F1 through F8)
2. Copy each new output
3. Open test_feedback_engineering.py in VS Code
4. Replace all 8 outputs in PIPELINE_OUTPUTS with the new responses
5. Save and run:

```
python test_feedback_engineering.py
```

```
✓ Evaluation completed 🎉! (time taken: 38.98s | token cost: 0.08882250000000001 USD)
» Test Results (4 total tests):
  » Pass Rate: 100.0% | Passed: 4 | Failed: 0

=====

» Want to share evals with your team, or a place for your test cases to live? ❤️🏠
» Run 'deepeval view' to analyze and save testing results on Confident AI.
```

What Should Have Improved

Metric	Before Fine-Tuning	After Fine-Tuning
Vague Feedback Handling	Mostly PASS, but some outputs assign CRITICAL severity to vague inputs and use inconsistent scales	Should fully PASS — consistent LOW severity, INVESTIGATION_NEEDED ticket type
Engineering Groundedness	Some FAIL — invents system names like 'API Gateway', 'GraphQL' not in original feedback	Should improve — uses only user-stated details
Engineering Usefulness	Mostly PASS	Should remain PASS or improve

Answer Relevancy	PASS	PASS
Severity consistency	Mixed scales (S1/S2/S3 vs P2/P3) across outputs	Consistent scale across all outputs
Overall pass rate	~25-50%	Should improve to 60-80%+

Checkpoint:

Your pass rate should have improved compared to the before-fine-tuning run. Save both screenshots side by side for your comparison.

Section 10: Troubleshooting

Problem	Fix
<code>pip install</code> fails with permission error	Try: <code>pip install --user deepeval openai python-dotenv</code>
<code>python</code> not recognized (Windows)	Reinstall Python from python.org and check "Add Python to PATH" during installation. Restart VS Code completely after installing.
<code>python</code> not recognized (Mac)	Use <code>python3</code> instead of <code>python</code> for all commands. macOS uses <code>python3</code> by default.
No module named <code>dotenv</code>	Run: <code>pip install python-dotenv</code> . This was included in Step 3.2 but may have been missed.
No module named <code>deepeval</code>	Run: <code>pip install deepeval openai python-dotenv</code> again. If you have multiple Python versions, try: <code>python3 -m pip install deepeval openai python-dotenv</code>

.env file not loading / "API key not found"	Make sure the file is named exactly <code>.env</code> (not <code>env.txt</code> or <code>.env.txt</code>). It must be in the same folder as the Python scripts. Open it and verify it contains <code>OPENAI_API_KEY=sk-proj-...</code> with no spaces around the <code>=</code> sign.
<code>insufficient_quota</code> error from OpenAI	Your OpenAI account needs billing credits. Add a payment method at platform.openai.com/account/billing .
DeepEval rate limit error	Wait 60 seconds and run again. The gpt-4o judge model has rate limits. If it keeps happening, wait 2-3 minutes between runs.
Evaluation scores are noisy or inconsistent	Make sure <code>test_feedback_engineering.py</code> uses <code>model="gpt-4o"</code> for all metric definitions. Using gpt-4o-mini as both the agent AND the judge produces unreliable scores.
n8n workflow import fails	Build manually: add Chat Trigger, two Agent nodes (Feedback Classifier and Engineering Insight Writer), and two OpenAI Chat Model sub-nodes (one connected to each agent). Copy prompts from the JSON file.
n8n Credential - Couldn't connect with these settings' Error	Please ignore for local model inference connection
Fine-tuning takes long time	Reduce the <code>max_steps</code> at line# 75 in the <code>fine_tune_training.py</code>
Fine-tuning script shows "validation failed"	Open the JSONL file and check that every line is valid JSON with exactly three messages: system, user, assistant. A common error is a missing comma or unclosed quote.
0% pass rate on all test cases	You may be using an older version of <code>test_feedback_engineering.py</code> that includes the <code>HallucinationMetric</code> . Use the updated file which uses only <code>Groundedness</code> , <code>Usefulness</code> , <code>Vague Handling</code> , and <code>Relevancy</code> .

Pass rate did not improve after fine-tuning	Verify you actually replaced the model IDs in both n8n LLM nodes. Run a vague input and check if the output says "Insufficient feedback for engineering action". If it still invents details, the old model is still being used.
---	--

Section 11: Key Takeaways

1. Evaluation before fine-tuning: You cannot improve what you cannot measure. DeepEval gives you a baseline score that tells you exactly where the model fails (inventing system names, fabricating reproduction steps).

2. Training data is the instruction set: The JSONL files are not random examples. They encode the exact behavior you want: how to classify, how to write engineering tickets, and critically, what to do when information is insufficient.

3. Two models, two behaviors: Each agent has a different job, so each needs different training data. The Classifier learns extraction patterns. The Insight Writer learns engineering writing patterns. Fine-tuning them separately means each can specialize.

4. Before/after comparison: Running the same test cases on the same metrics before and after fine-tuning proves the improvement is real and measurable, not subjective.

5. The evaluation-optimization flywheel: Evaluate, find failures, fix (via prompt or fine-tuning), re-evaluate, confirm improvement. This is the cycle every PM/TPM shipping AI features should follow.

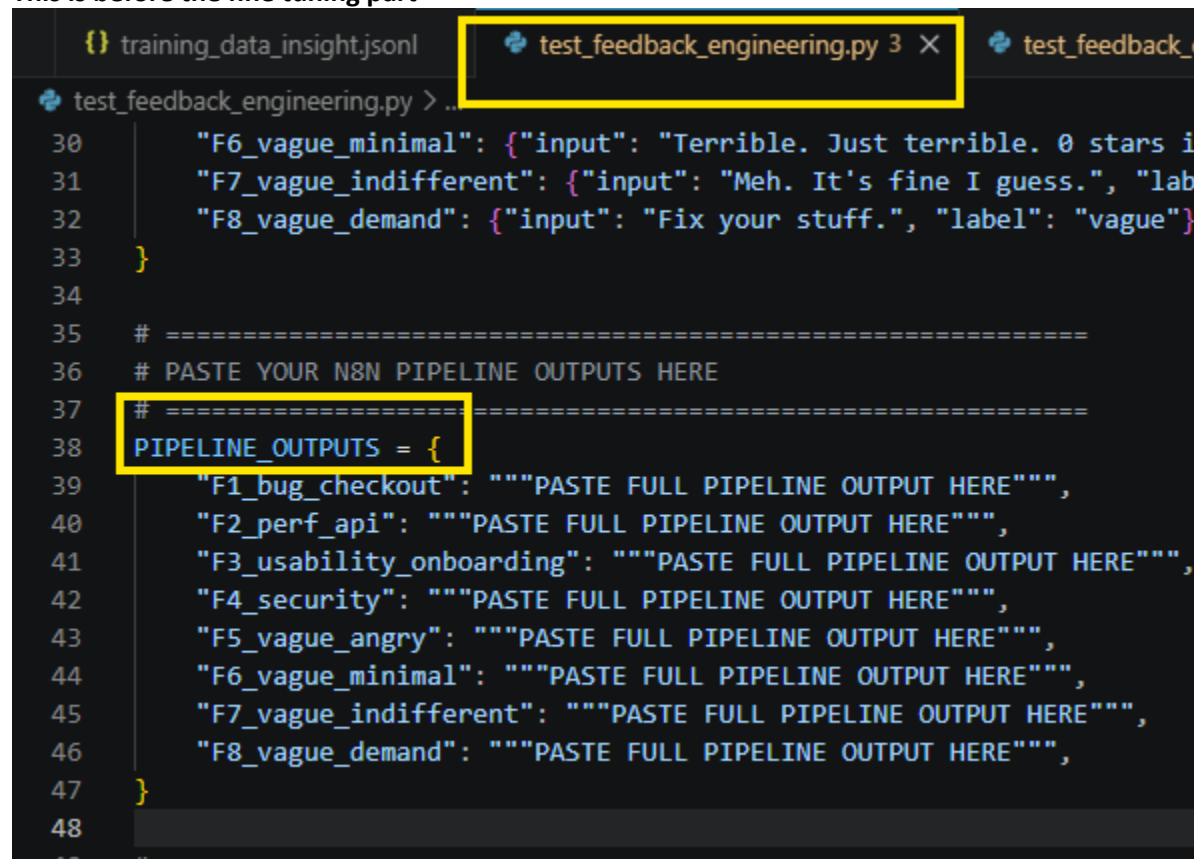
Appendix - 1

Note about these base-model outputs: These outputs demonstrate several errors that fine-tuning is designed to fix. Pay attention to two specific issues:

1. **F5 has SEVERITY: S1-CRITICAL** for a completely vague input ("This app is garbage"). The training data teaches the model to assign LOW severity to vague inputs since there is no information to justify high severity. After fine-tuning, this should change to LOW.
2. **The severity scale is inconsistent across outputs.** F1 uses S3-Moderate, F2 uses S2-HIGH, F3 uses P3-MEDIUM, F4 uses P2-HIGH, F5 uses S1-CRITICAL, F6 uses S2-LOW, F7 uses S3-UNKNOWN, and F8 uses S5-INFO. This is the base model inventing different scales for each output. After fine-tuning, the model should use a consistent severity scale across all outputs. This inconsistency is one of the clearest improvements you will see after fine-tuning.

Paste this in test_engineering_feedback.py

This is before the fine tuning part



```
training_data_insight.json test_feedback_engineering.py 3 X test_feedback_e
test_feedback_engineering.py > ..
30     "F6_vague_minimal": {"input": "Terrible. Just terrible. 0 stars i
31     "F7_vague_indifferent": {"input": "Meh. It's fine I guess.", "lab
32     "F8_vague_demand": {"input": "Fix your stuff.", "label": "vague"}
33 }
34
35 # =====
36 # PASTE YOUR N8N PIPELINE OUTPUTS HERE
37 # =====
38 PIPELINE_OUTPUTS = {
39     "F1_bug_checkout": ""PASTE FULL PIPELINE OUTPUT HERE"",
40     "F2_perf_api": ""PASTE FULL PIPELINE OUTPUT HERE"",
41     "F3_usability_onboarding": ""PASTE FULL PIPELINE OUTPUT HERE"",
42     "F4_security": ""PASTE FULL PIPELINE OUTPUT HERE"",
43     "F5_vague_angry": ""PASTE FULL PIPELINE OUTPUT HERE"",
44     "F6_vague_minimal": ""PASTE FULL PIPELINE OUTPUT HERE"",
45     "F7_vague_indifferent": ""PASTE FULL PIPELINE OUTPUT HERE"",
46     "F8_vague_demand": ""PASTE FULL PIPELINE OUTPUT HERE"",
47 }
48
49 #
50 PIPELINE_OUTPUTS = {
```

```
"F1_bug_checkout": ""=== ENGINEERING INSIGHT ===
TITLE: Checkout freeze on iOS Safari when applying discount codes
TICKET_TYPE: BUG
SEVERITY: S3-Moderate
```

TECHNICAL SUMMARY: Checkout process stalls completely when applying discount codes on iOS Safari. The freeze occurs consistently on the latest iOS version (user did not specify version) on an iPhone 14 device. This may indicate a JavaScript execution hang or rendering issue specific to Safari.

AFFECTED COMPONENTS:

System: Checkout UI

Platform: iOS Safari

Environment: Production web, mobile

REPRODUCTION STEPS:

INSUFFICIENT DATA - steps below are what we know, not confirmed reproduction steps

Open the checkout page in Safari on iOS

Enter and apply a discount code

Checkout page freezes completely

INVESTIGATION CHECKLIST:

[] Test discount code application on iOS Safari with various code lengths and formats

[] Check if Safari's JavaScript execution or rendering API calls are blocking

[] Review recent changes to the checkout page JS/CSS that may impact Safari rendering

DATA GAPS FOR ENGINEERING:

Specific iOS version, JavaScript console errors, network request logs, execution time of the discount apply function.

PRIORITY RECOMMENDATION: P2-HIGH

PRIORITY JUSTIFICATION: Single user report but critical path checkout stop. Browser platform and mobile context elevate priority. Check for any regressions in Safari-specific code paths."",

"F2_perf_api": """"=== ENGINEERING INSIGHT ===
TITLE: API response time degradation from p95 200ms to 800ms - Enterprise customer
TICKET_TYPE: BUG
SEVERITY: S2-HIGH

TECHNICAL SUMMARY: The Enterprise plan API for bigcorp.com is experiencing a latency regression where the p95 response time has increased from 200ms to 800ms. This indicates a systemic performance issue likely affecting the entire API surface for this account, not isolated endpoints. The customer has three open tickets about this, indicating a sustained degradation.

AFFECTED COMPONENTS:

System: API Gateway

Platform: All (API v2, GraphQL, webhook delivery)

Environment: Production

REPRODUCTION STEPS:

INSUFFICIENT DATA - steps below are what we know, not confirmed reproduction steps

Call any API endpoint in the Enterprise suite

Measure p95 latency

Compare against historical benchmarks (200ms, now 800ms)

INVESTIGATION CHECKLIST:

- [] Review API Gateway performance dashboards for bigcorp.com
- [] Identify which API endpoints have the largest latency increases
- [] Check backend service latencies for correlated spike in response time

DATA GAPS FOR ENGINEERING:

Details on which endpoints are 800ms (API Gateway logs can reveal)

Traffic volume metrics for api-team@bigcorp.com over the last month

Load balancer request queue times

Database query performance for API data-fetching calls

PRIORITY RECOMMENDATION: P2-HIGH

PRIORITY JUSTIFICATION: High-severity latency regression on an Enterprise customer API. Affects production systems, sustained for weeks, three open tickets. Blocker for bigcorp.com's production workloads. Blast radius is API service degradation - likely all endpoints for this account.

```
""",  
  "F3_usability_onboarding": """"=== ENGINEERING INSIGHT ===
```

TITLE: Confusing onboarding flow - 404 in getting started guide, outdated UI in tutorials

TICKET_TYPE: INVESTIGATION_NEEDED

SEVERITY: P3-MEDIUM

TECHNICAL SUMMARY: The user is unable to create a project after completing account signup due to multiple onboarding friction points, including a broken 404 link in the official guide and discrepancies between the current UI and third-party tutorial content. They are on the free plan using Chrome on Windows.

AFFECTED COMPONENTS:

System: Web application onboarding flow

Platform: Chrome on Windows

Environment: Free plan, Windows web client

REPRODUCTION STEPS:

INSUFFICIENT DATA - steps below are what we know, not confirmed reproduction steps

Signed up for a free plan

Attempted to follow the official getting started guide

Encountered a 404 link in the guide

Watched 3 YouTube tutorials but found all of them show a different UI

INVESTIGATION CHECKLIST:

[] Validate the 404 link in the current getting started guide

[] Identify any recent UI changes that could have broken tutorial content

[] Review user flow from account creation to first project - where do users typically drop off?

DATA GAPS FOR ENGINEERING:

Which step in the onboarding flow the user gets stuck on - they didn't specify how far they got after signup

The exact URL of the 404 and which guide it was in

Whether the UI they see differs from other users on the same plan

PRIORITY RECOMMENDATION: P3-MEDIUM

PRIORITY JUSTIFICATION: Onboarding issues directly impact user activation and retention. URL breakage is a clear fix, and if tutorials are obsolete, that's a broader content maintainability issue. But this is one report so far, not a confirmed cohort-wide bug.

""",

"F4_security": """"=== ENGINEERING INSIGHT ===

TITLE: Unauthorized login on account AC-41209 - data access audit needed

TICKET_TYPE: BUG

SEVERITY: P2-HIGH

TECHNICAL SUMMARY: An unauthorized login was detected on account lisa.t@company.com from IP 203.0.113.42 (Romania) at 3:47 AM EST. The user reports they did not initiate the session. The user has since changed their password and enabled 2FA, but they need an audit of whether data was accessed during the session.

AFFECTED COMPONENTS:

System: Authentication/Session Management

Platform: Web Application

Environment: Production

REPRODUCTION STEPS:

INSUFFICIENT DATA - steps below are what we know, not confirmed reproduction steps

Unauthorized login detection triggered (time: 3:47 AM EST)

IP address: 203.0.113.42 (geolocation: Romania)

Account email: lisa.t@company.com

User changed password and enabled 2FA post-incident (not part of the unauthorized flow)

INVESTIGATION CHECKLIST:

[] Query auth logs for successful and failed login attempts on account AC-41209 in the last 30 days

[] Check session activity metadata (accessed resources, API calls made, data exports)

[] Cross-reference IP 203.0.113.42 against known proxies/VPNs and any other accounts that logged in from that IP

DATA GAPS FOR ENGINEERING:

The authentication log itself: session duration, whether cookies were generated

Whether any sensitive data (files, API keys) was accessed during the session

Whether the IP address has been used for previous suspicious logins across other accounts

Whether authentication was via password (credentials compromise) or session hijacking (token theft)

PRIORITY RECOMMENDATION: P2-HIGH

PRIORITY JUSTIFICATION: User accounts compromised through unauthorized access represent a security incident. The request for a 30-day audit indicates potential data exposure, which escalates the impact. Time-to-detection and remediation (password change, 2FA) are not engineering's fault, but audit scope is still urgent.

""",

"F5_vague_angry": """"=== ENGINEERING INSIGHT ===

TITLE: User complaint - issue type and technical details UNKNOWN

TICKET_TYPE: INVESTIGATION_NEEDED

SEVERITY: S1-CRITICAL

TECHNICAL SUMMARY: Insufficient feedback for engineering action - no specific issue, component, or behavior described.

AFFECTED COMPONENTS:

System: UNKNOWN

Platform: UNKNOWN

Environment: UNKNOWN

REPRODUCTION STEPS:

INSUFFICIENT DATA - no steps provided, cannot infer anything technical.

INVESTIGATION CHECKLIST:

[] Reach out to user for specific issue, platform, and technical details

[] Tag for follow-up if they respond with a tech issue

[] Squad lead to decide if escalation to engineering is warranted

DATA GAPS FOR ENGINEERING:

Literally everything - no component, no error, no context. Cannot assign to a team without a technical signal.

PRIORITY RECOMMENDATION: P4-LOW

PRIORITY JUSTIFICATION: Single data point, zero technical content. Engineering cannot act until user specifies what is broken.

```
""",
  "F6_vague_minimal": """"=== ENGINEERING INSIGHT ===
TITLE: User dissatisfaction - no specific problem reported
TICKET_TYPE: INVESTIGATION_NEEDED
SEVERITY: S2-LOW
```

TECHNICAL SUMMARY: Insufficient data for engineering action. The user expresses frustration with the product but does not describe what is broken or what platform/environment is affected.

AFFECTED COMPONENTS:

System: UNKNOWN

Platform: UNKNOWN

Environment: UNKNOWN

REPRODUCTION STEPS:

INSUFFICIENT DATA - no steps provided.

INVESTIGATION CHECKLIST:

- [] Reach out to user for details on what product feature is failing
- [] Ask for platform/browser/environment details to scope investigation
- [] Request screenshots or error messages to diagnose the issue

DATA GAPS FOR ENGINEERING:

Product feature

Technical platform

Environment (browser/device, OS)

Expected vs. actual behavior

PRIORITY RECOMMENDATION: P4-LOW

PRIORITY JUSTIFICATION: No actionable technical information for engineering. Can't investigate until features, platforms, and errors are clarified."",

"F7_vague_indifferent": ""=== ENGINEERING INSIGHT ===

TITLE: No actionable technical information

TICKET_TYPE: INVESTIGATION_NEEDED

SEVERITY: S3-UNKNOWN

TECHNICAL SUMMARY: Insufficient feedback for engineering action. No issue or feature is described.

AFFECTED COMPONENTS:

System: UNKNOWN

Platform: UNKNOWN

Environment: UNKNOWN

REPRODUCTION STEPS:

INSUFFICIENT DATA - no steps provided

INVESTIGATION CHECKLIST:

[] Reach out to user for technical details

[] Request affected component, platform, and steps to reproduce

[] If technical details gathered, escalate back to engineering

DATA GAPS FOR ENGINEERING:

Everything. No system, no steps, no error messages, no platform.

PRIORITY RECOMMENDATION: P4-LOW

PRIORITY JUSTIFICATION: No issue described. Engineering cannot investigate or prioritize.

""",
"F8_vague_demand": """"=== ENGINEERING INSIGHT ===
TITLE: Unclassified feedback - no technical issue described
TICKET_TYPE: UNCATEGORIZED
SEVERITY: S5-INFO

TECHNICAL SUMMARY: This ticket does not contain any information about a technical issue, bug, or feature request. Engineering cannot take action without details.

AFFECTED COMPONENTS:

System: UNKNOWN

Platform: UNKNOWN

Environment: UNKNOWN

REPRODUCTION STEPS:

INSUFFICIENT DATA - no steps provided

INVESTIGATION CHECKLIST:

[] Reach out to user for technical details

DATA GAPS FOR ENGINEERING:

Everything - not a report about anything technical.

PRIORITY RECOMMENDATION: P4-LOW

PRIORITY JUSTIFICATION: No engineering signal, no investigation possible.

""",